# Operational Technology (OT) Railroad Security Assessment Study Guide

This study guide prepares you for the OT Railroad North security assessment. It covers OT network mapping, PLC API command manipulation, and traffic analysis. Read each section before starting the corresponding modules. The examples use dummy IP addresses and fictional routes. This teaches you the methods without leaking the actual flags. No em-dashes are used in this document.

## Table Of Contents

# Phase 1: Reconnaissance (Modules 1 to 3)

This phase covers mapping the OT network and finding PLC endpoints.

## Conceptual Background

Operational Technology (OT) networks control physical machinery rather than processing data. In this lab, the OT network coordinates physical railway track switches and signals. The architecture uses a Master-Slave Programmable Logic Controller (PLC) design. The Master PLC receives status updates from the Slave PLCs. It also coordinates commands to prevent train collisions. However, OT systems often use legacy protocols with no built-in security. Identifying exposed devices and APIs is the first step in a security audit.

## Key Commands Reference

Use these commands on your workstation terminal to locate targets.

| Command | Purpose | Key Flags | Example |
|---|---|---|---|
| `nmap` | Scan ports on target hosts | `-p` scan specific ports; `-F` fast scan | `nmap -p 502,8085 10.0.0.1` |
| `curl` | Query HTTP REST endpoints | `-s` silent mode (hides progress meters) | `curl -s http://10.0.0.1:8085/status` |
| `jq` | Pretty-print and parse JSON | `.` formats output (requires piping) | `curl -s http://10.0.0.1:8085/status` |

## Core Concepts Deep-Dive

### Modbus TCP (Port 502)

Modbus TCP is an industrial communications protocol. It has no built-in authentication, encryption, or access controls. Any device on the network can send Modbus packets to read or write PLC register values. Standard Modbus TCP operations listen on port 502.

### Industrial REST APIs

Modern PLCs often expose web portals or REST APIs on standard HTTP ports. These portals allow administrators to check firmware details, network parameters, and system health. If these

APIs do not implement authentication, attackers can query system statistics. This exposes the state of the physical processes.

# Worked Examples

## Example 1: Scan for Modbus TCP and API services

You suspect a PLC is located in the IP range. You run a port scan targeting the Modbus port and the custom management API port.

```
# Scan a specific target IP
nmap -p 502,8085 172.99.0.10

# Output:

# PORT     STATE SERVICE
# 502/tcp  open  mbap
# 8085/tcp open  unknown
```

This confirms both the industrial protocol and the web API are accessible.

## Example 2: Check system status via API

You query the unauthenticated status API to discover how many safety mechanisms are active.

```
# Fetch and print the JSON status payload
curl -s http://172.99.0.10:8085/api/status | python3 -m json.tool

# Output:
# {
#   "plc_state": "RUN",
#   "safety_interlocks": [
#     {"name": "JunctionLock", "active": true},
#     {"name": "SpeedGovernor", "active": false}
#   ]
# }
```

In this dummy example, there is 1 active safety interlock.

# Before You Start These Modules

1. What port does the Modbus TCP protocol use by default?
2. Why is authentication often absent in industrial control systems?

3. How can you pretty-print JSON responses in a terminal without installing external tools? (Hint: Use `python3 -m json.tool`).

## Common Mistakes

- **Omitting the port number.** The REST API runs on a custom port (8085), not the default HTTP port (80). Ensure you specify the port in your `curl` command.
- **Running aggressive scans on OT networks.** OT devices can crash when scanned too quickly. Avoid using aggressive timing flags (`-T5`) in real-world environments.

## Further Reading

- `man nmap` (Network exploration and port scanning manual)
- `man curl` (HTTP transfer tool reference)
- ISA/IEC 62443 standard guidelines (Security for industrial automation systems)

# Phase 2: Exploitation (Modules 4 to 7)

This phase covers sending unauthorized commands, fuzzing inputs, and testing safety interlocks.

## Conceptual Background

Because the Master PLC lacks authentication, any network node can write command parameters. This allows attackers to switch tracks or override alerts. However, industrial systems often implement safety interlocks. These interlocks reject commands that would cause physical collisions or damage. Security assessments test whether these interlocks can be bypassed or abused. They also test how the system handles invalid input (fuzzing) or high command volumes.

## Key Commands Reference

| Command | Purpose | Key Flags | Example |
|---|---|---|---|
| `curl` | Send command payloads | `-X POST` specify method; `-H` add headers; `-d` send data | `curl -X POST -H "Content-Type: application/json" -d '{"param": 1}' http://10.0.0.1/api` |
| `seq` | Generate a sequence of numbers | Used in shell loops to repeat tasks | `seq 1 10` |

## Core Concepts Deep-Dive

### JSON Command Payloads

The control API processes commands sent as JSON payloads inside POST requests. The payload specifies the targeted hardware component and the state change. For example, it might specify a track segment and a target route.

```
{

  "segment_id": 1,
  "route": "ROUTE_A"
}
```

### Alarm Flooding (Denial of Service)

Industrial operators rely on SCADA dashboards to respond to system alerts. If an attacker floods the controller with thousands of rapid commands, the log queues can overflow. This blinds the operators and prevents them from noticing critical physical warnings.

### Input Fuzzing

Fuzzing sends invalid, unexpected, or random data to an API endpoint. It tests how the controller handles bad inputs. If the API crashes instead of returning a clean error validation code, it represents a security vulnerability.

## Worked Examples

### Example 1: Reroute a segment

You send a POST request to change the target route on segment 5.

```
# Send track command
curl -s -X POST http://172.99.0.10:8085/api/command -H "Content-Type:
application/json" -d '{"segment_id": 5, "route": "ROUTE_X"}'

# Output:
# {"status": "success", "requested_route": "ROUTE_X"}
```

### Example 2: Flood the system with commands

You use a loop to send 20 rapid command requests. This simulates an alarm flooding attack.

```
for val in $(seq 1 20); do
  curl -s -X POST http://172.99.0.10:8085/api/command \
    -H "Content-Type: application/json" \
    -d '{"segment_id": 9, "route": "ROUTE_Y"}' > /dev/null
done
```

## Before You Start These Modules

1. What HTTP method is typically used to submit commands to a REST API?
2. What are industrial safety interlocks, and how do they protect machinery?
3. How does sending an invalid route parameter help test API validation?

## Common Mistakes

- **Malformatted JSON syntax.** Ensure you use single quotes around the `-d` argument. Use double quotes inside the JSON string itself.
- **Ignoring the response payload.** Read the return messages carefully. They specify whether the command succeeded or was blocked by a safety interlock.

## Further Reading

- RFC 8259 (The JavaScript Object Notation (JSON) Data Interchange Format)
- OWASP API Security Top 10 (Improper Assets Management guidelines)

# Phase 3: Forensics and Defense (Modules 8 to 10)

This phase covers traffic analysis, emergency recoveries, and memory inspection.

## Conceptual Background

Security operations require continuous monitoring and emergency response. When an incident occurs, network analysts capture packets using tools like Wireshark. They analyze the traffic to reconstruct the attacker's actions. If the system enters a failed safety state, recovery keys must be sent to restore service. Finally, advanced assessments search for hidden debug parameters. These parameters may be left exposed in the controller's active memory.

## Forensics Tools and Techniques

- **Wireshark**: A network packet analyzer. It captures and displays network traffic in real time. It allows you to follow TCP streams to read the raw data sent over the network.
- **Emergency Clearances**: APIs that clear emergency stops. These are used by operators to restore control after a fault is resolved.
- **Memory Inspection**: Querying configuration variables. This reveals hidden flags or debug configurations left by developers.

## Worked Examples

### Example 1: View raw HTTP request packets

You open Wireshark and capture a command packet. You follow the TCP stream to view the raw request:

```
POST /api/command HTTP/1.1
Host: 172.99.0.10:8085
Content-Type: application/json
{"segment_id": 2, "route": "ROUTE_B"}
```

This confirms the exact parameter key used to target the segment is `segment_id`.

### Example 2: Clear an emergency stop

The system is locked down due to an emergency condition. You send a clear command to restore normal operation.

```
# Trigger emergency clear API
curl -s -X POST http://172.99.0.10:8085/api/emergency/clear
```

```
# Output:
# {"status": "cleared", "action": "emergency_cleared"}
```

## Before You Start These Modules

1. Which Wireshark filter isolates HTTP POST requests? (Hint: `http.request.method == "POST"`).
2. What is the difference between an emergency stop state and a normal running state?
3. Where can developer debug flags be located during a system audit?

## Common Mistakes

- **Using HTTP instead of HTTPS for Wireshark.** The Wireshark interface uses secure HTTPS. If you type `http://localhost:3001` instead of `https://localhost:3001`, the page will not load.
- **Failing to wait for system initialization.** The PLC network requires up to 60 seconds to boot. Ensure the heartbeat system is active before trying to clear emergency stops.

## Further Reading

- Wireshark User Guide: https://www.wireshark.org/docs/wsug_html/
- Modbus TCP Security Guidelines (SANS Industrial Control Systems Security reference)